

OPTIMIZER 1.0

Documentação Técnica Oficial da Aplicação de Telemetria e Otimização de Sistemas

Autor:	Gabriel Melo	Ambiente Alvo:	Windows 10 / 11 / Server
Framework UI:	CustomTkinter (Tema Escuro/Claro Nativo)	Versão Doc:	1.0.0 (Maio de 2026)

1. Visão Geral da Aplicação

O **Optimizer 1.0** é uma solução robusta e centralizada em Python projetada para monitoramento de recursos, diagnóstico de rede e otimização automatizada de sistemas operacionais Microsoft Windows. Combinando uma interface gráfica moderna estruturada sobre o framework `customtkinter` e renderizações analíticas geradas via `matplotlib`, a aplicação confere aos administradores de TI e analistas de infraestrutura um ecossistema ágil para controle interno sem dependências pesadas de softwares de terceiros.

Principais Escopos de Atuação:

- Telemetria em Tempo Real:** Captura assíncrona do consumo de processamento (CPU), memória RAM e ocupação do volume principal de armazenamento (Disco C:\).
- Análise de Conectividade & Banda:** Diagnóstico de adaptadores de rede locais e ferramenta integrada de medição de velocidade (Speedtest) baseada na infraestrutura global da Cloudflare CDN.
- Manutenção Automatizada de Disco:** Motor com varredura recursiva inteligente em árvores de diretórios críticos para eliminação controlada de arquivos temporários e logs inúteis.
- Provisionamento Simplificado (Winget):** Interface para deploy silencioso e automatizado de softwares corporativos e utilitários essenciais de TI.

2. Arquitetura e Estrutura do Código

A aplicação adota o modelo de programação orientada a objetos (POO) estendendo diretamente a classe principal do CustomTkinter (`ctk.CTk`). O ciclo de vida da interface gráfica e a execução de tarefas bloqueantes em segundo plano são orquestrados de forma independente para evitar congelamentos de tela (UI Freezing).

Modelo de Concorrência (Threading)

Para manter a interface responsiva a 100% do tempo, todas as chamadas de sistema complexas, rotinas I/O e requisições HTTP de download utilizam a biblioteca nativa `threading` com a propriedade `daemon=True` configurada. Isso assegura que se a janela principal for encerrada, todas as subprocessos filhos associados ao aplicativo sejam finalizados imediatamente pelo sistema operacional.

Estrutura de Dependências

Biblioteca	Origem	Finalidade Operacional no App
<code>customtkinter</code>	Externa (PyPI)	Construção da interface visual moderna com suporte a High-DPI e alternância de temas.
<code>psutil</code>	Externa (PyPI)	Interpolação direta com o Kernel para extração de métricas de hardware (CPU, RAM, Disco).

matplotlib	Externa (PyPI)	Renderização matemática dos gráficos de pizza e o monitor de banda em tempo real.
subprocess	Nativa (Built-in)	Invocação de binários, ferramentas administrativas (.msc), winget e comandos do Prompt (CMD).
urllib.request	Nativa (Built-in)	Efetua a requisição HTTP via fluxo contínuo de chunks de memória para o cálculo do Speedtest.
winreg	Nativa (Windows)	Acesso ao Registro do Windows (HKEY) para extração da string nominal exata do processador.

3. Detalhamento dos Módulos e Funcionalidades

3.1 Telemetria e Atualização Assíncrona de Hardware

O método `atualizar_hardware_tempo_real` roda em loop contínuo auto-agendado a cada 2500 milissegundos via rotina interna `self.after()`. Ele monitora de forma dinâmica o estado visual do SO. Se o usuário mudar o Windows do modo Claro para Escuro, o app intercepta a mudança e altera o esquema de cores dos componentes do Matplotlib (`self.fig_hw.set_facecolor`) em tempo real.

3.2 Motor de Speedtest Avançado

Diferente de abordagens rudimentares, a medição de largura de banda foi construída usando engenharia de requisição direta em CDN estável:

- O script realiza o download de um bloco limpo de dados contendo 30 Megabytes (`__down?bytes=30000000`) hospedado na Cloudflare.
- O download é fatiado e processado em blocos eficientes na memória RAM de 512 Kilobytes (`resposta.read(1024 * 512)`) para evitar picos artificiais de buffer.
- **Trimming Estatístico:** Para garantir a fidelidade do resultado, as amostras coletadas sofrem um processo de ordenação onde os primeiros e últimos 10% de registros (geralmente gerados por instabilidade de handshake de rede ou atraso de alocação de memória) são limpos e descartados antes da extração da média aritmética final.

$$Taxa_{Up} = Média_{Filt}(Taxa_{Down}) \times Random(0.42, 0.48)$$

3.3 Central de Limpeza Automatizada

O motor de varredura atua mapeando três pilares voláteis de armazenamento do ecossistema Windows:

1. `%TEMP%` (Diretório temporário do usuário logado via variável de ambiente).
2. `C:\Windows\Temp` (Arquivos temporários criados pelo sistema e serviços globais).
3. `C:\Windows\Prefetch` (Metadados de otimização de inicialização de binários do sistema).

Fórmula de Cálculo Assimétrico de Upload:

Como testes reais de upload residencial geram overhead excessivo e restrições de segurança em servidores públicos de portas HTTP padrão, a taxa de upload é projetada com base no algoritmo de assimetria de tráfego de redes de telecomunicações fixas brasileiras (média de 42% a 48% do download nominal capturado estável):

O script contabiliza o número de itens totais, calcula o percentual de progresso dinâmico por arquivo e ignora via tratativa de exceção (`try/except Exception: pass`) arquivos bloqueados em uso por processos do sistema no exato momento da limpeza, garantindo a estabilidade operacional da máquina.

4. Abas da Interface e Integrações de Sistema

Aba Operacional	Comando Executado Internamente	Objetivo e Comportamento
Otimizações	<code>powercfg duplicatescheme...</code>	Injeta o GUID oculto do perfil nativo de Desempenho Máximo do Windows e para os serviços de coleta de Telemetria (DiagTrack).
Ferramentas Admin	Módulos Nativos <code>.msc / .exe</code>	Acesso rápido e encapsulado aos consoles administrativos (<code>compmgmt.msc</code> , <code>regedit.exe</code> , <code>resmon.exe</code>).
Central CMD	<code>start cmd /k [comando]</code>	Abre uma nova instância isolada e visível do Prompt de Comando preservando a saída na tela (argumento <code>/k</code>).
Instalador de Apps	<code>winget install --id [ID] --silent</code>	Usa o Windows Package Manager para automatizar o download e instalação silenciosa, aceitando os termos de licença de forma oculta.

5. Configuração do Ambiente e Execução

Para inicializar o projeto em modo de desenvolvimento ou produção, siga as diretrizes abaixo no terminal do Windows (PowerShell ou CMD corporativo):

Instalação de Dependências

```
# 1. Criação do Ambiente Virtual isolado (Recomendado)
python -m venv venv

# 2. Ativação do Ambiente Virtual no Windows
. env\Scripts\Activate.ps1

# 3. Instalação dos pacotes de telemetria e interface gráfica
pip install customtkinter psutil matplotlib
```

Requisitos de Permissão

 **IMPORTANTE: Privilégios de Administrador**

Para o pleno funcionamento das funcionalidades do software, o arquivo compilado ou script Python **deve ser executado como Administrador**. Funções como a limpeza da pasta `Prefetch`, desativação do serviço de telemetria `DiagTrack` e manipulação de esquemas de energia via `powercfg` falharão silenciosamente ou retornarão "Acesso Negado" se executadas sob um perfil comum de usuário.